AI Image Generator

3. System Architecture

Version 1.0
22 October 2026

Project owner: Frazile AI
Platform: NVIDIA GeForce RTX 3090, Windows 11, Python 3.12

Stable Diffusion XL + Real-ESRGAN, served locally with Gradio, with per-image GPU telemetry recorded in SQLite.

3. System Architecture

3.1 Component overview

The application is a single Python process. Gradio serves the browser interface and calls plain Python functions; those functions use five project modules and two GPU-resident models.

Component	File	Responsibility
Web application	app.py	Builds the Gradio UI, owns the loaded models, orchestrates one generation end to end, renders the details report, serves the History tab.
Prompt parser	prompt_parser.py	Turns free text into a Settings object: size, steps, guidance, count, seed, style, upscale target. Pure function, no GPU.
Upscaler	upscaler.py	Wraps Real-ESRGAN through spandrel; tiled x4 passes; resizes or crops to the requested target.
GPU telemetry	gpu_info.py	Static GPU facts from torch; live temperature, power, clocks, fan and load from nvidia-smi.
Database	db.py	SQLite schema, migration, insert, fetch, search, per-row get, aggregate stats, command-line viewer.
CLI generator	generate.py	Standalone batch generation without the web UI; the original script from increment 1.

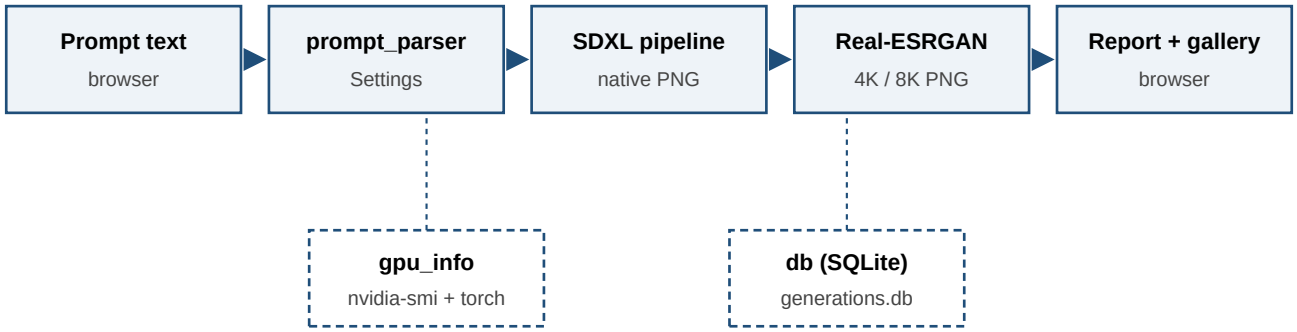

Figure 3.1: Generation pipeline. Solid boxes are the per-image data path; dashed boxes are sampled or written around steps 3 and 4.

3.2 Generation pipeline

For every image the sequence is:

1. The prompt text is parsed into a Settings object. Numeric directives are stripped; style words add descriptor suffixes.
2. The seed is fixed (from the prompt or random) and a CUDA generator is created.
3. Peak-memory counters are reset and a telemetry snapshot is taken (temperature, power, clocks).
4. The SDXL pipeline runs for the requested number of steps at the native size. A per-step callback drives the progress bar.
5. CUDA is synchronised, the inference time is recorded, and a second telemetry snapshot is taken while the GPU is still under load.
6. The native image is saved as PNG.
7. If an upscale target is set, the Real-ESRGAN stage runs: one or two tiled x4 passes, then a Lanczos resample to the target, then an optional centre crop to an exact size. The upscaled PNG is saved.
8. A record with about 50 fields is inserted into generations.db.

9. The final image (upscaled if applicable) and a Markdown report are returned to the browser.

3.3 Data flow and storage

Data	Where it lives	Format	Written by
Native images	outputs/YYYYMMDD-HHMMSS_seedN.png	PNG, 1 to 2 MB	app.py, generate.py
Upscaled images	outputs/..._seedN_3840px.png	PNG, 8 to 31 MB	app.py
Generation records	generations.db	SQLite, one table	db.py
Model weights	Hugging Face cache, models/	safetensors, pth	downloaded once
Logs	stdout of the app process	text	app.py

3.4 Design decisions

Prompt as the only control surface. Early versions had dropdowns for style and size and sliders for steps and guidance. These were removed in favour of a parser because a single text box is faster to use, is self-documenting through the live interpretation line, and makes every generation reproducible from the prompt string alone, which is stored in the database.

Two-stage resolution. SDXL produces incoherent output above roughly 1 megapixel and attention cost grows with the square of pixel count. Generating at native size and upscaling with a super-resolution network is the standard approach, gives better results, and completes in a few seconds. The parser treats "4k" and "8k" as output-size requests, not as native resolutions.

Tiled upscaling. Real-ESRGAN on a full 3840 px image would need tens of gigabytes for intermediate feature maps. Processing 512 px tiles with a 32 px overlap bounds memory at a few hundred megabytes and hides seams.

Telemetry via nvidia-smi. Shelling out to nvidia-smi avoids a dependency on NVML Python bindings, works on every driver, and costs about 100 ms per call. Two calls per image is negligible against a 10 s generation.

SQLite over a server database. A single-user desktop tool needs no server. SQLite is in the standard library, the file is trivially backed up, and any SQL client can open it. The connect() helper adds missing columns so the schema can grow without migrations scripts.

Models stay resident. Loading SDXL takes about 30 s. Both models are loaded once at process start and held on the GPU for the lifetime of the server, so each generation pays only inference cost.

3.5 Concurrency model

Gradio runs the generate function in a worker thread and queues additional requests. The GPU is used by one generation at a time. The History tab reads the database on its own thread; SQLite handles the concurrent read safely because each call opens its own short-lived connection.